

Otimização do Desempenho de uma Simulação de Dinâmica de Fluidos - Parte 2

1st Augusto Campos
PG57510

2st João Rodrigues
PG52687

3st Tiago Silva
PG57617

Abstract—Este trabalho visa melhorar o desempenho de uma simulação de dinâmica de fluidos por meio da paralelização do código utilizando OpenMP. Após identificar os *hotspots* mais críticos em termos de tempo de execução com a ferramenta de *profiling perf*, implementamos estratégias de paralelismo em loops computacionalmente intensivos. As otimizações realizadas permitiram uma redução significativa no tempo de execução, tirando partido de múltiplos núcleos de processamento, sem comprometer a correção e fiabilidade do programa.

Index Terms—otimização de desempenho, profiling, OpenMP, threads, paralelismo de memória partilhada

I. INTRODUÇÃO

O objetivo desta fase foi aplicar técnicas de paralelismo para otimizar um programa, reduzindo o seu tempo de execução. Para atingir este objetivo, utilizámos ferramentas de *profiling* para identificar os pontos críticos do programa, onde o custo computacional é mais elevado. De seguida, implementámos técnicas de paralelismo e avaliamos o desempenho da solução adotada por meio de análises de escalabilidade e eficiência.

II. IDENTIFICAÇÃO DOS HOTSPOTS DO PROGRAMA

Através da ferramenta de *profiling perf*, foi possível identificar que a função *lin_solve* é responsável pela maior parte do tempo de execução do programa. Por este motivo, esta função foi escolhida como o principal foco de paralelização, uma vez que melhorar o seu desempenho pode trazer uma melhoria significativa no tempo de execução do programa.

III. IMPLEMENTAÇÃO DE TÉCNICAS DE PARALELISMO

Nesta implementação, optamos por paralelizar as operações usando loops paralelizados com OpenMP, aproveitando instruções como *#pragma omp parallel for* e *collapse* para distribuir uniformemente o trabalho entre as threads. Também utilizamos *reduction* para computar valores acumulados (neste caso o *max_c* em *lin_solve*) de forma eficiente em paralelo e *simd* para explorar o paralelismo a nível de vetor em algumas operações internas. O uso de barreiras (*#pragma omp barrier*) foi necessário em pontos específicos para sincronizar as threads entre fases distintas da computação, garantindo a consistência dos dados.

Outra abordagem possível seria o uso de tasks, em que o trabalho seria dividido dinamicamente em tarefas de tempo de execução imprevisível, permitindo um agendamento mais adaptável.

A escolha dos loops paralelizados foi motivada pela natureza do problema e pela estrutura regular dos dados. Neste

código, a matriz permite uma divisão estática eficiente das iterações. Ao usar loops paralelizados com *collapse* e *reduction*, garantimos que cada thread executa blocos contínuos de trabalho, reduzindo o overhead associado ao *scheduling* dinâmico e aproveitando melhor a cache.

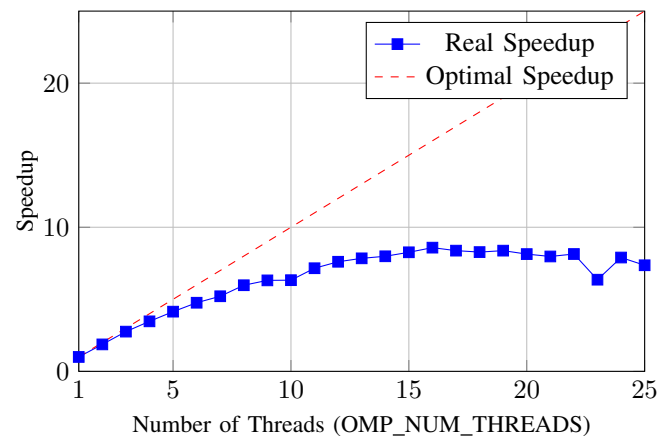
Por outro lado, a abordagem baseada em tasks, embora mais flexível, introduz maior overhead devido ao custo de criação, gestão e sincronização das tarefas. Essa técnica seria mais vantajosa em problemas com carga de trabalho altamente irregular ou dinâmica, o que não é o caso aqui.

Portanto, a nossa abordagem é superior neste contexto mantendo o código eficiente, simples e escalável em sistemas com múltiplos núcleos, enquanto minimiza o overhead e aproveita a estrutura regular da matriz para uma execução paralela previsível e balanceada.

IV. ANÁLISE DE PERFORMANCE

Como podemos ver pelo gráfico da Figura 1, a redução do tempo de execução é significativa até cerca de 16 threads, demonstrando uma boa escalabilidade inicial. Para valores superiores, os ganhos tornam-se marginais, e surgem oscilações no tempo de execução.

Estas oscilações podem ser explicadas devido ao overhead de comunicação entre threads, conflitos de cache e barreiras de sincronização, que afetam a eficiência à medida que mais threads são utilizadas. Apesar disso, o desempenho geral mostra que a paralelização foi bem sucedida, proporcionando uma redução substancial no tempo de execução.



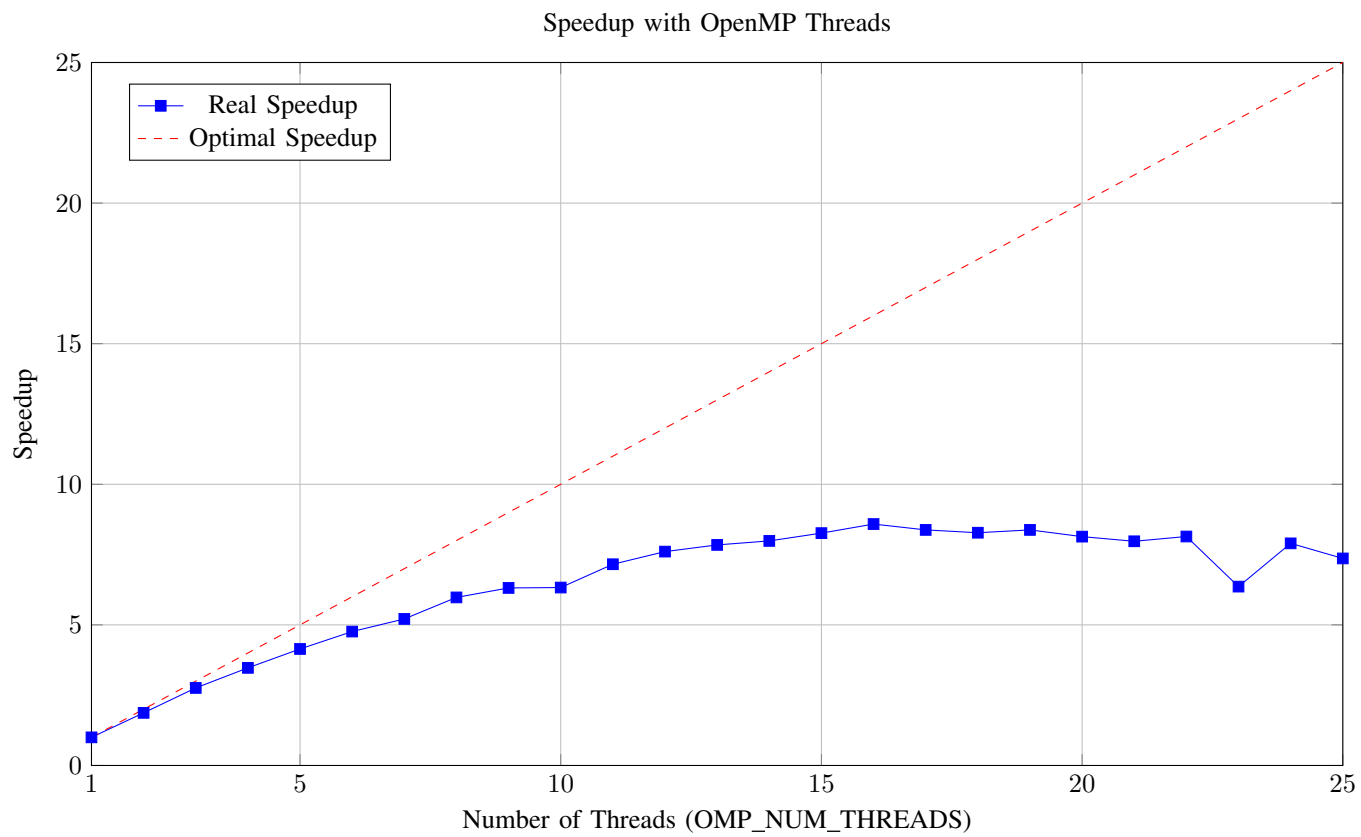


Fig. 1. Speedup with OpenMP Threads